

# **Lecture 8: A Deep Dive Into Linear Bandits**

**ID 6002W: Online and Reinforcement Learning**

Web-enabled M.Tech. program, IIT Madras

# Outline of Today's Class

- Understanding the mathematics of the **linear bandits model**
- Understanding the UCB algorithm in this model
- Understanding the real-world interpretation and flexibility of the model

Philosophy for today:

let us understand **what** changes we are making to basic bandit model,  
then we will revisit **why** we are doing so

# Recap: Basic Bandit Formalism

- We have a fixed set of  $K$  arms
- At each time  $t$ , we can pull any one arm  $A_t$
- Pulling an arm gives us a random reward  $X_t$
- Assumption: the rewards from any fixed arm are i.i.d.
- Reward distribution of arm  $i$  is  $\nu_i$  (fixed, but unknown)
- Mean reward of arm  $i$  is  $\mu_i$  (fixed, but unknown)



# Recap: Basic Bandit Formalism

- The best arm is  $i^* = \arg \max_{i \in [K]} \mu_i$
- The goal is to minimise the cumulative regret over  $T$  rounds:

$$R(T) = \sum_{t=1}^T \left( \mu^* - \mathbb{E}[\mu_{A_t}] \right)$$

- Here, we take expectation over  $A_t$ , because actions can depend on previous rewards, which are random

# Recap: Basic Bandit Formalism

- A good algorithm, like UCB or Thompson Sampling, gets sublinear regret:

$$R(T) \sim \sum_{i \in [K]} \frac{\log T}{\Delta_i}$$

- Getting sublinear regret involves a careful trade-off of exploration and exploitation
- Regret scales linearly with number of arms
  - We must try each arm a few arms to know whether it is good or not

# Adding Structure to Multi-Armed Bandits

We now make a couple of new assumptions to the basic MAB framework

In the **linear bandits model**, we assume:

- each arm  $i$  has a *context vector*  $x_i \in \mathbb{R}^d$  (fixed and known)
- the reward distribution has the following structure:

$$\nu_i = \mathcal{N}(\mu_i, 1)$$

$$\mu_i = \langle \theta, x_i \rangle$$

Where  $\theta \in \mathbb{R}^d$  is fixed, but unknown



# Adding Structure to Multi-Armed Bandits

## Understanding the assumptions

The reward distributions across different arms are related through  $\theta$ :  $\mu_i = \langle \theta, x_i \rangle$

- Suppose  $\theta$  was known, in addition to  $x_i$ 
  - Then we know  $\mu_i$  for each arm  $i \Rightarrow$  no problem left to solve!
- Suppose  $\theta$  was known, but  $x_i$  was unknown
  - Then we go back to basic multi-armed bandits, where  $\mu_i$  have no relation

So what change does the new model ( $\theta$  unknown,  $x_i$  known) bring?

# Notation

- $x_i$  : context vector for arm  $i$ ;  $i \in 1, \dots, K$
- $x(t)$  : context vector of arm played at time  $t$ ;  $t \in \{1, 2, \dots, T\}$ 
  - $x(t) = x_{A_t}$
- $x^k$  :  $k$ -th component of a context vector  $x$ ;  $k \in \{1, 2, \dots, d\}$ 
  - $x_i \in \mathbb{R}^d$ ,  $x(t) \in \mathbb{R}^d$ ,  $x^k \in \mathbb{R}$

We will also introduce a new notation for reward:

- $y(t)$ : reward after pulling an arm in round  $t$  (earlier, this was  $X_t$ )



# Advantage of Context Information

- Consider a setting with  $K = 5$ ,  $T = 5$ ,  $d = 2$
- Suppose the goal is best arm identification (only for this example)
- Simple strategy: play each arm once, report the arm with best reward
- With context vectors, can we do better?

1 data point for each  
unknown quantity  $\mu_i \in \mathbb{R}$

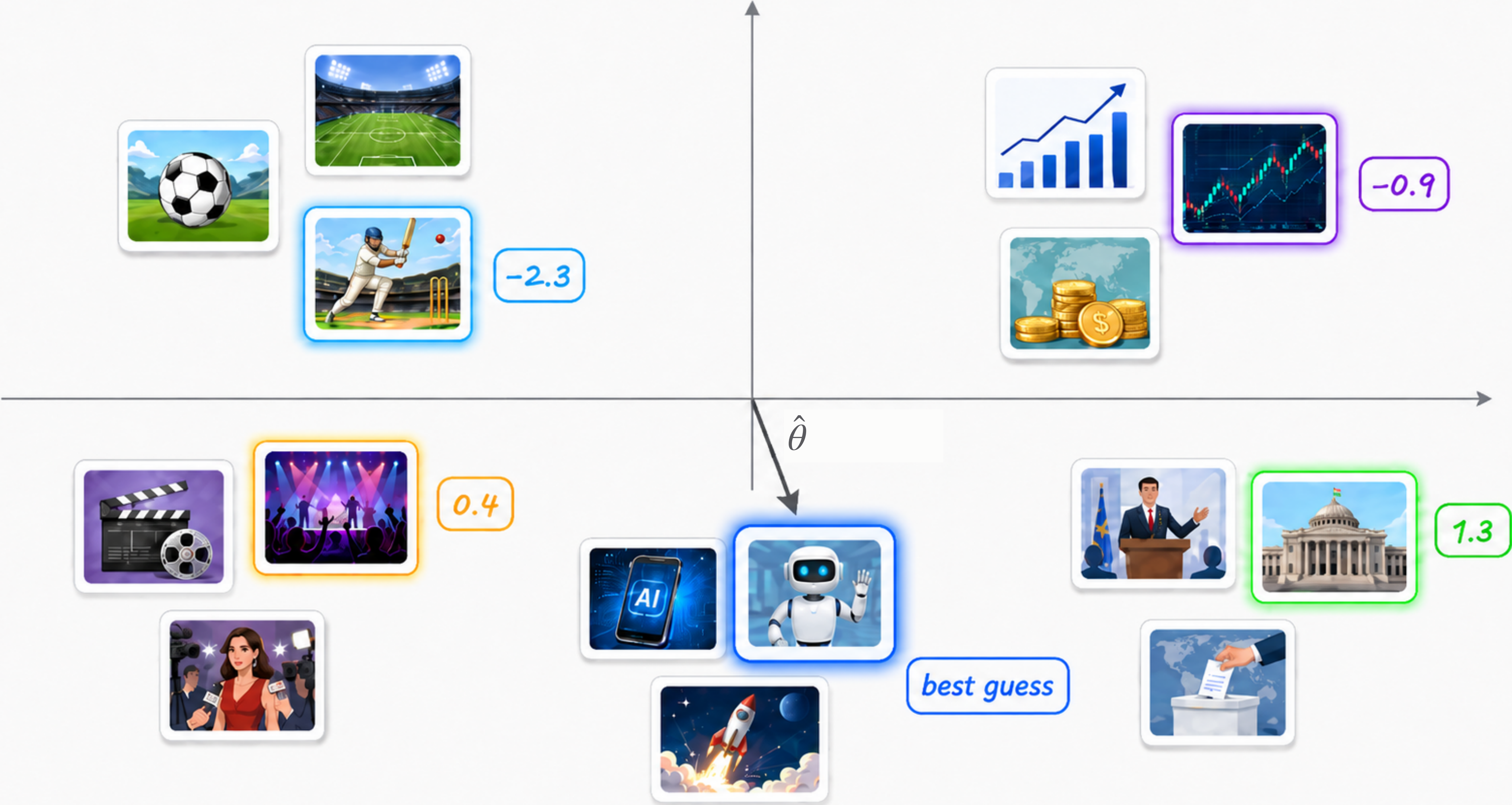
- We can try to *estimate*  $\hat{\theta}$  from data:

10 data points for a single  
unknown quantity  $\theta \in \mathbb{R}^2$

$$(x(1), y(1)), (x(2), y(2)), \dots (x(10), y(10))$$

- Using the estimate, we can calculate  $\hat{i} = \arg \max_{i \in [K]} \langle \hat{\theta}, x_i \rangle$

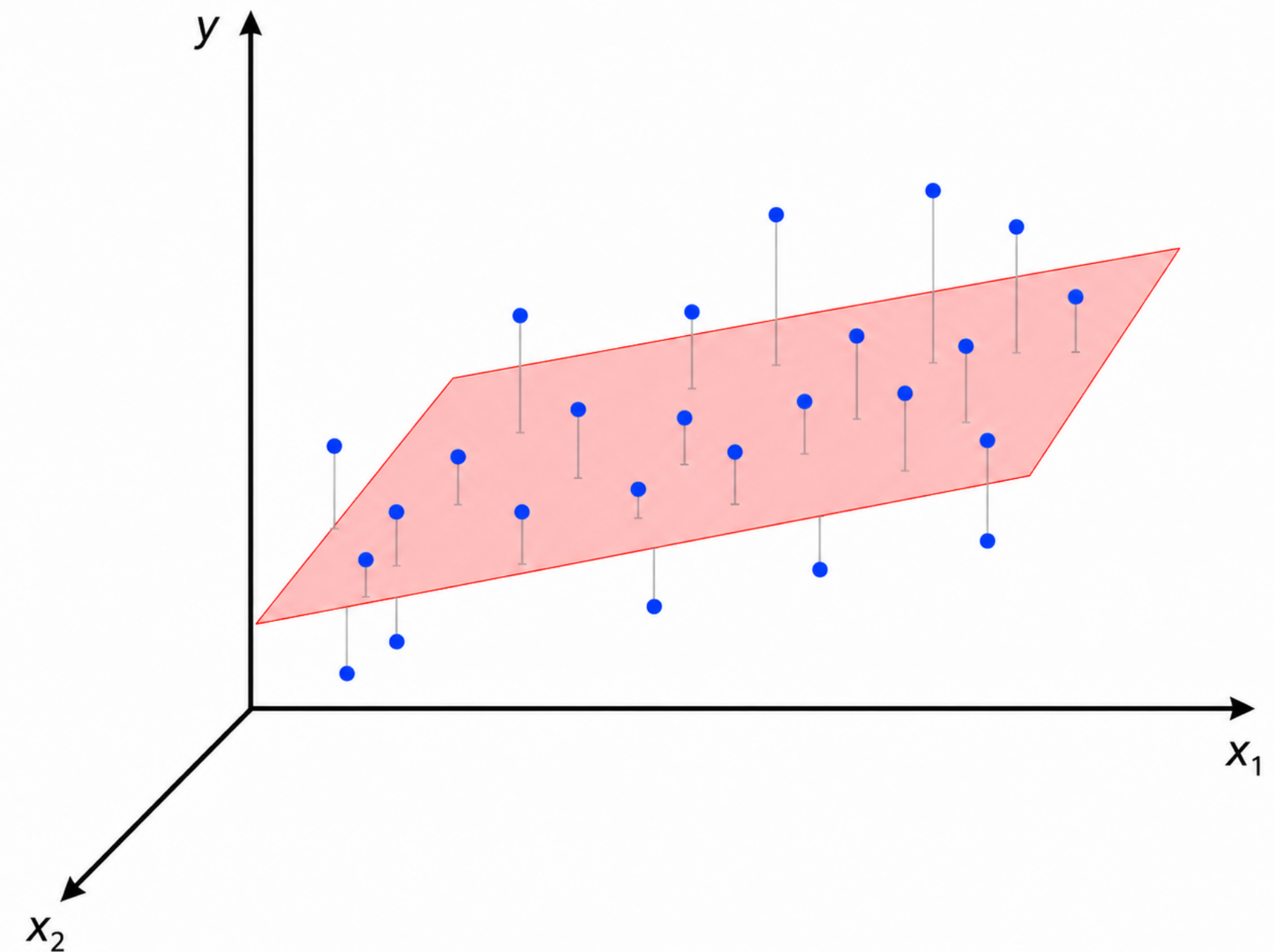
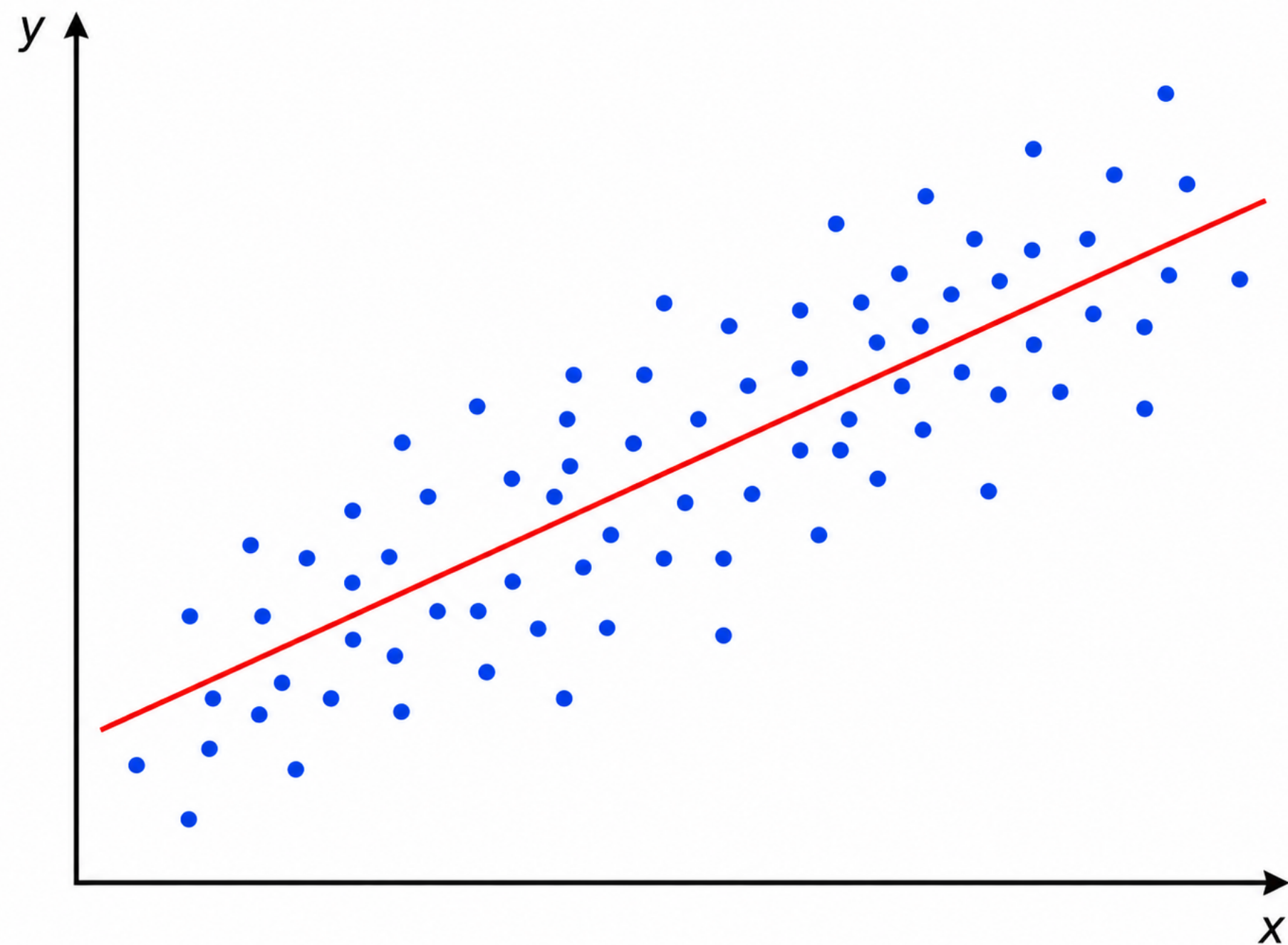
# Revisiting Previous Example





# Estimating $\theta$

- How should one calculate  $\hat{\theta}$ ? The problem is identical to linear regression
- Data:  $(x(1), y(1)), (x(2), y(2)), \dots, (x(T), y(T))$  where  $y(t) = \langle \theta, x(t) \rangle + \eta_t$



# Estimating $\theta$

- How should one calculate  $\hat{\theta}$ ? The problem is identical to linear regression
- Data:  $(x(1), y(1)), (x(2), y(2)), \dots (x(T), y(T))$  where  $y(t) = \langle \theta, x(t) \rangle + \eta_t$
- Estimate  $\theta$  by solving the least-squares problem

$$\hat{\theta} = \arg \min_{\theta} \sum_{t=1}^T (y(t) - \langle \theta, x(t) \rangle)^2$$

- For regret minimisation, we need to make a smart decision at each time  $t$ 
  - We need to calculate this estimate repeatedly

# A Closed Form Solution

At any time  $t$ , based on the data available, the estimate can be calculated as

$$\hat{\theta}_t = V_t^{-1} b_t$$

$$\hat{\theta}_t = \arg \min_{\theta} \sum_{s=1}^t (y(s) - \langle \theta, x(s) \rangle)^2$$

$$V_t = \sum_{s=1}^t x(s)x(s)^{\top}$$

$$b_t = \sum_{s=1}^t x(s)y(s)$$

- $V_t \in \mathbb{R}^{d \times d}$ ,  $b_t \in \mathbb{R}^d$
- Both can be easily incremented with every new data point
- All operations are matrix-vector operations—easy. Computing inverse?



# Introducing a Regulariser

- When number of datapoints  $t$  is less than  $d$ , the matrix  $V_t$  will not be invertible!
  - For  $V_t \in \mathbb{R}^{d \times d}$  to be invertible, the dimension of its column space should be  $d$
  - For any matrix of the form  $\sum_{s=1}^T x(s)x(s)^\top$ , the column space is the span of  $\{x(1), \dots, x(t)\}$
  - $t$  vectors cannot span a  $d$ -dimensional space (if  $t < d$ )
- Solution: we add a **regulariser** to the least-squares problem

$$\hat{\theta}_t = \arg \min_{\theta} \sum_{t=1}^T (y(s) - \langle \theta, x(s) \rangle)^2 + \lambda \|\theta\|^2$$

- $V_t = \lambda I + \sum_{s=1}^t x(s)x(s)^\top$ , which is always invertible

# Towards a Bandit Algorithm

- We have  $K$  arms, each arm  $i$  with a known context vector  $x_i$
- Mean rewards follow the structure  $\mu_i = \langle \theta, x_i \rangle$  for some unknown  $\theta \in \mathbb{R}^d$
- We can calculate an estimate  $\hat{\theta}_t$  at each step  $t$
- $\hat{\theta}_t$  gives us an estimate of the arm rewards, and hence a guess for the best arm
- Using this estimate, we can act greedily at each step  $t$ :  $A_{t+1} = \arg \max_{i \in [K]} \langle \hat{\theta}_t, x_i \rangle$
- But greedy strategy, or even  $\varepsilon$ -greedy, is not good for regret minimisation!

# Towards UCB for Linear Bandits

- UCB algorithm requires an uncertainty measure
- For basic bandits, we were estimating  $\mu_i$ 
  - We maintained confidence intervals around  $\hat{\mu}_i$
  - with the guarantee:  $\mu_i \in [\hat{\mu}_i - \epsilon, \hat{\mu}_i + \epsilon]$  with high probability
- In linear bandits, we are estimating  $\theta \in \mathbb{R}^d$
- We need a set  $\mathcal{C}$ , centred around  $\hat{\theta}$ , such that  $\theta \in \mathcal{C}$  with high probability



# Confidence Ellipsoids

- In high dimensions, we represent confidence sets using ellipsoids

$$\mathcal{C}_t = \left\{ \theta \in \mathbb{R}^d : \|\theta - \hat{\theta}_t\|_{V_t}^2 \leq \beta \right\}$$

Why is this an ellipsoid?

- $\|\theta\|_2^2 = \theta^\top \theta = \theta^\top I \theta$
- $\|\theta\|_2^2 \leq \beta$  denotes a ball around the origin
- $\|\theta - \hat{\theta}\|_2^2 \leq \beta$  denotes a ball around  $\hat{\theta}$

# Confidence Ellipsoids

- In high dimensions, we represent confidence sets using ellipsoids

$$\mathcal{C}_t = \left\{ \theta \in \mathbb{R}^d : \|\theta - \hat{\theta}_t\|_{V_t}^2 \leq \beta \right\}$$

Why is this an ellipsoid?

- $\|\theta\|_V^2 = \theta^\top V \theta$
- If  $V$  is a diagonal matrix,  $\|\theta\|_V^2 = V_{1,1}(\theta^1)^2 + \dots + V_{d,d}(\theta^d)^2$
- So  $\|\theta\|_V^2 \leq \beta$  is an ellipsoid around origin

radius along axis  $i$  :  $\sqrt{\beta/V_{i,i}}$

# Confidence Ellipsoids

- In high dimensions, we represent confidence sets using ellipsoids

$$\mathcal{C}_t = \left\{ \theta \in \mathbb{R}^d : \|\theta - \hat{\theta}_t\|_{V_t}^2 \leq \beta \right\}$$

Why is this an ellipsoid?

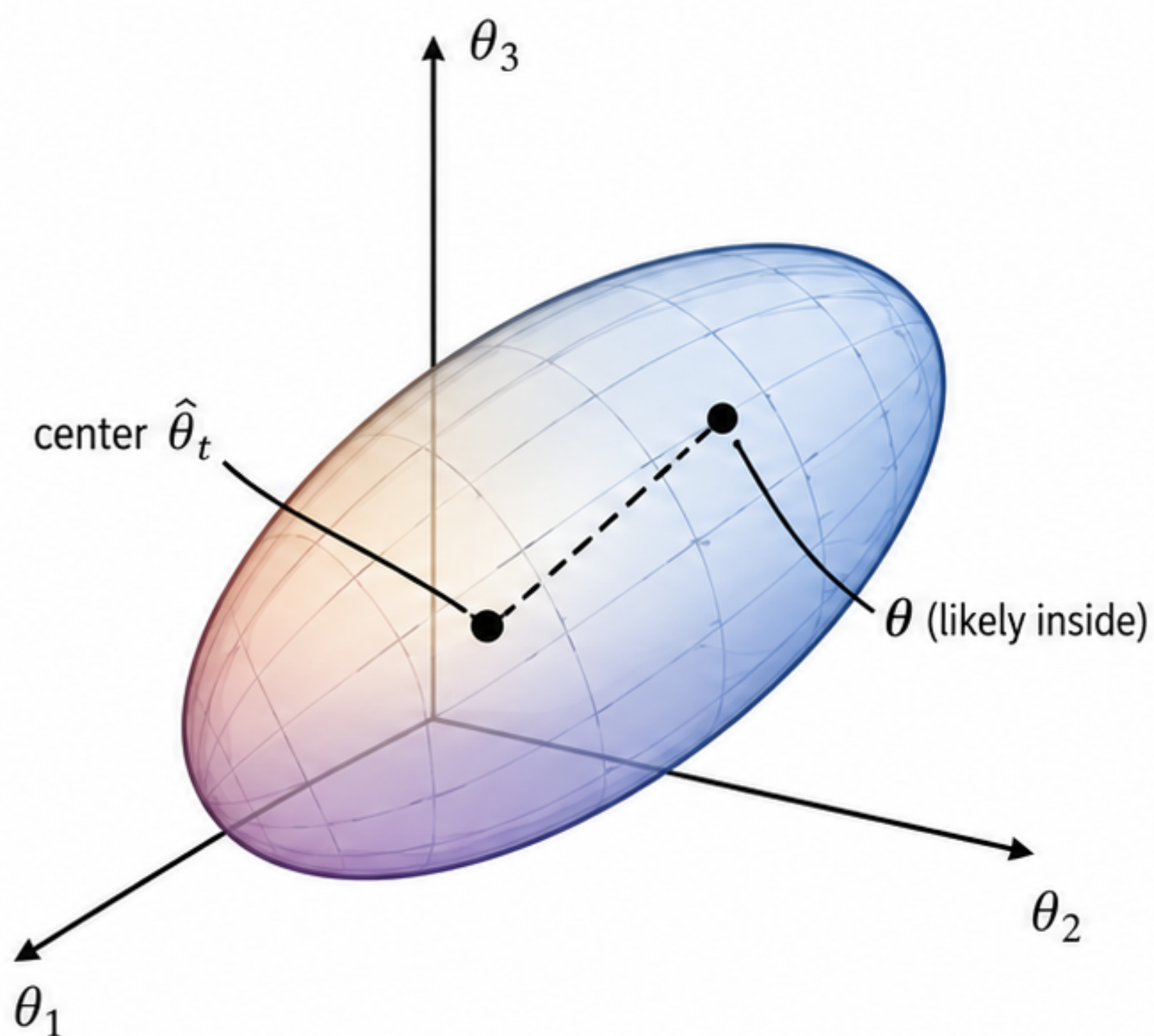
- In general,  $V_t = \sum_{s=1}^t x(s)x(s)^\top = \sum_{k=1}^d \lambda_k v_k v_k^\top$  ← **Eigenvalue decomposition for symmetric matrices**

- $\|\theta\|_V^2 \leq \beta$  is an ellipsoid with axes aligned along  $(v_1, \dots, v_d)$  and radius



# Visual Representation

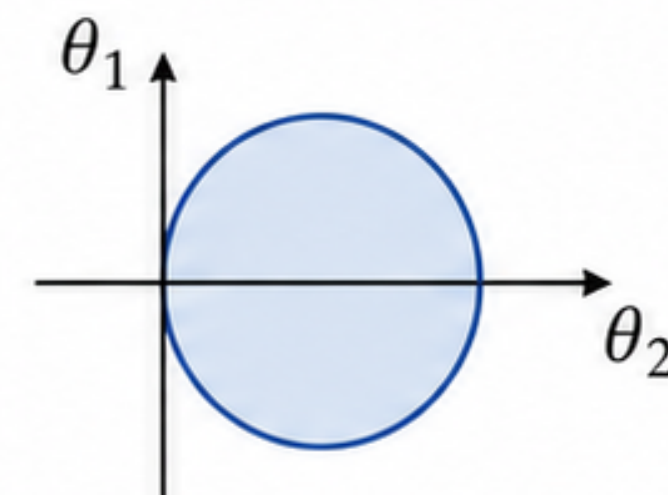
## Confidence ellipsoid



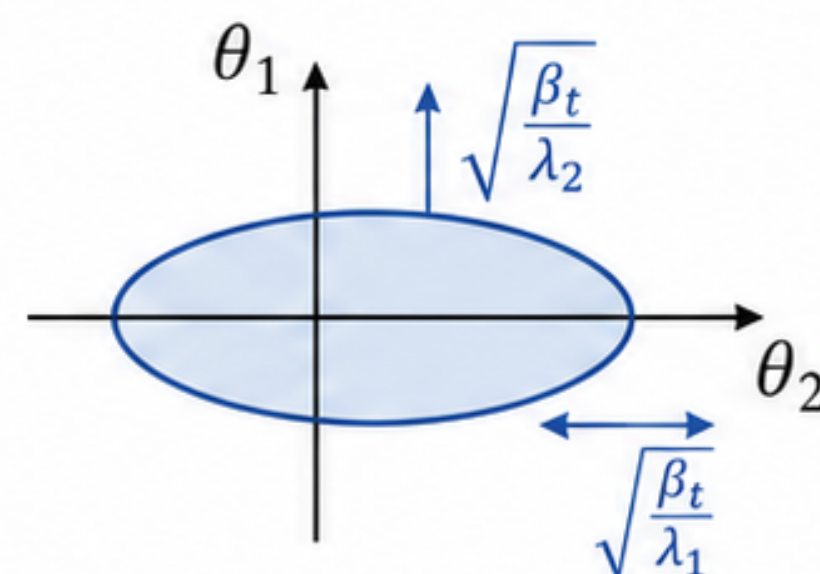
$$(\theta - \hat{\theta}_t)^T V_t (\theta - \hat{\theta}_t) \leq \beta_t$$

$$V_t \succ 0, \beta_t > 0$$

## How $V_t$ shapes the set

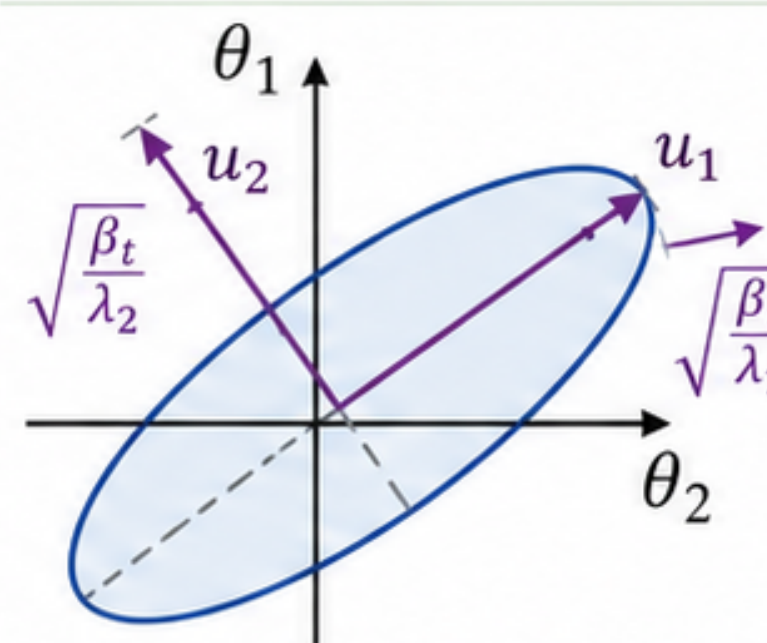


- ①  $\lambda_1 = \lambda_2$   
equal uncertainty  
in all directions



- ②  $V_t = \text{diag}(\lambda_1, \lambda_2)$

larger  $\lambda_i \rightarrow$  shorter axis  
smaller  $\lambda_i \rightarrow$  longer axis



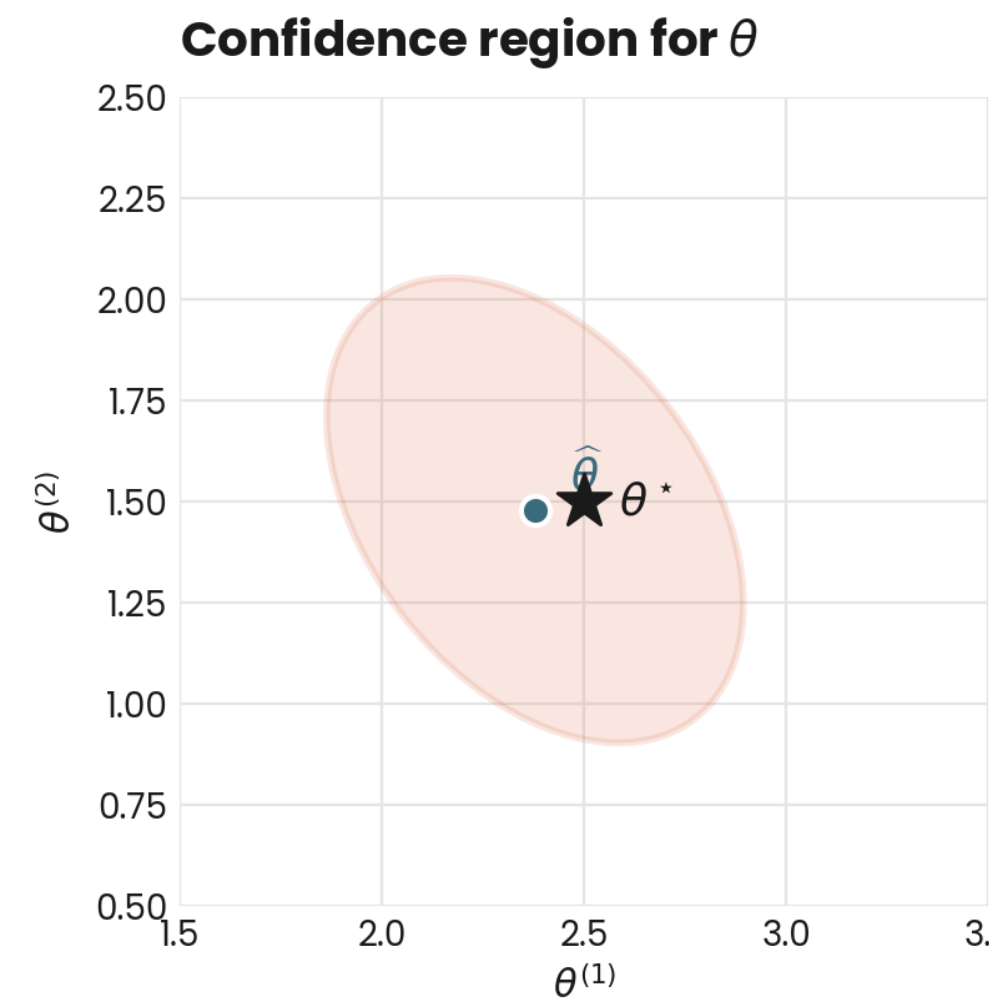
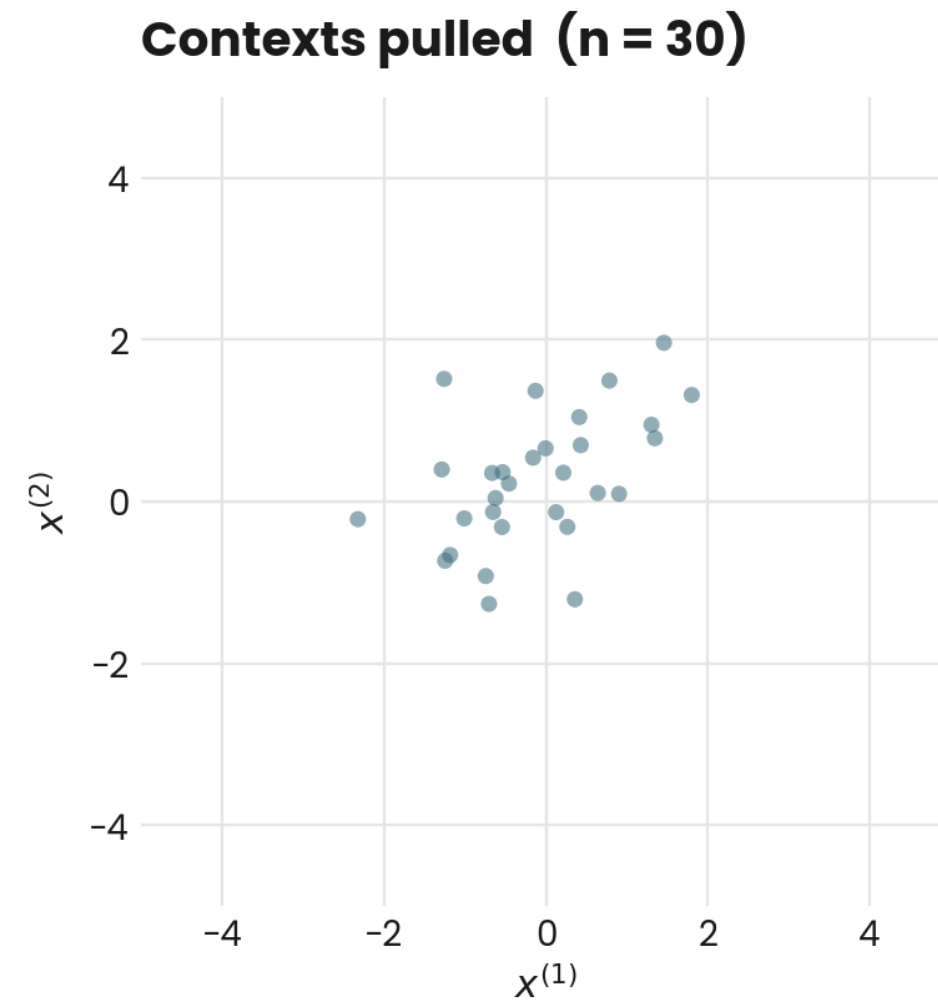
- ③  $V_t = U \text{diag}(\lambda_1, \lambda_2) U^T$   
eigenvectors set orientation

Larger eigenvalue  $\rightarrow$  more information  $\rightarrow$  shorter axis.

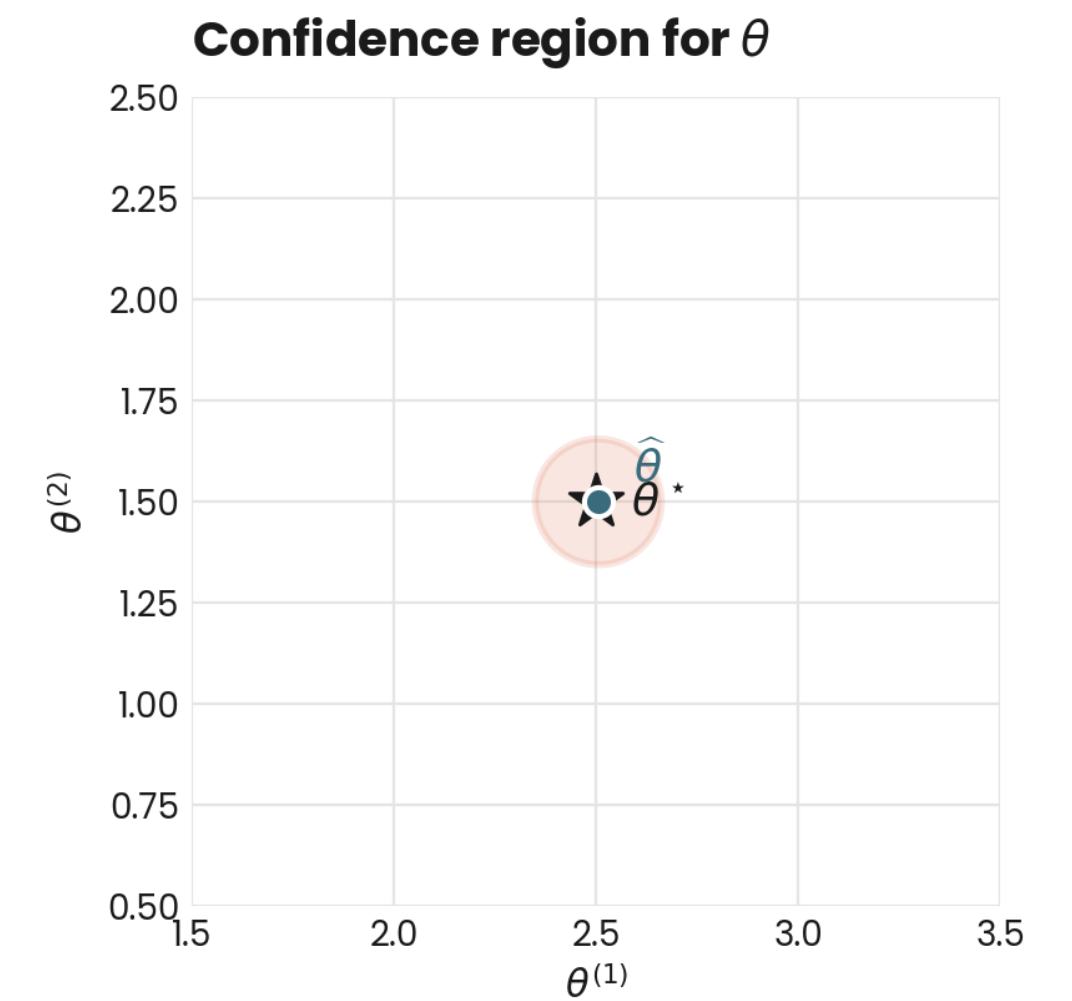
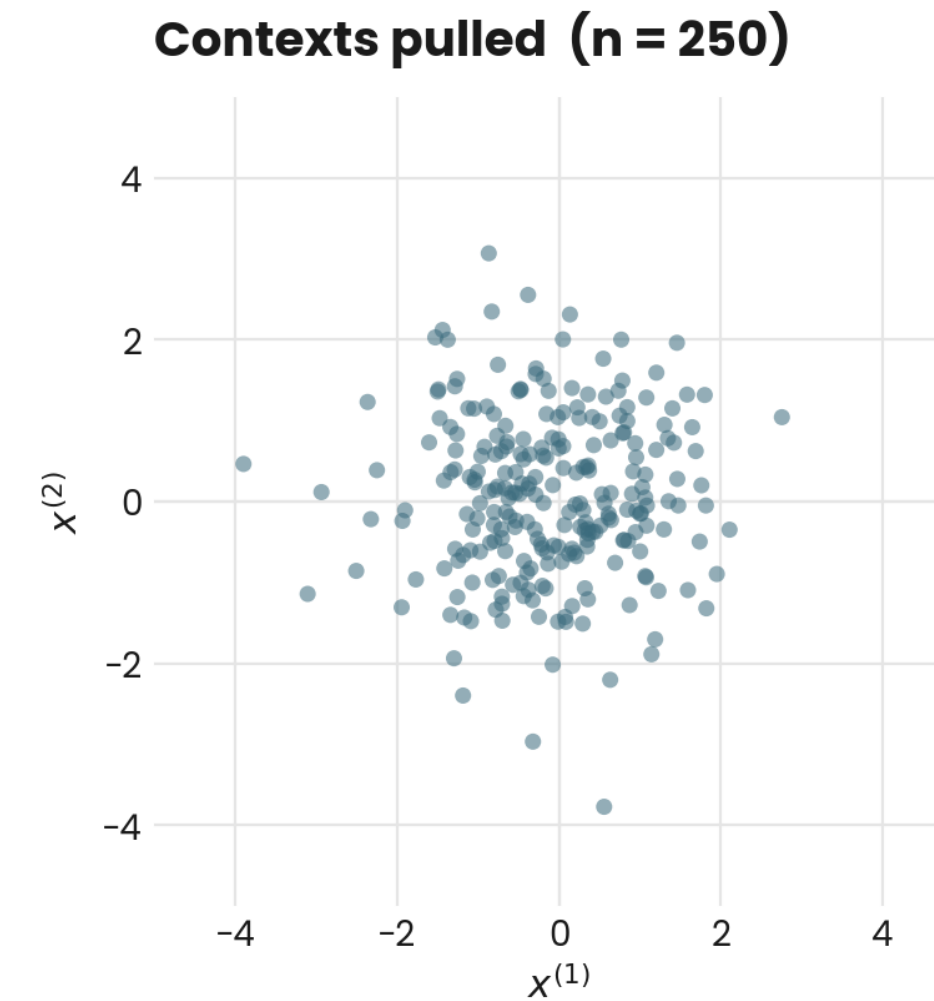


# Confidence Ellipsoids — Different Settings

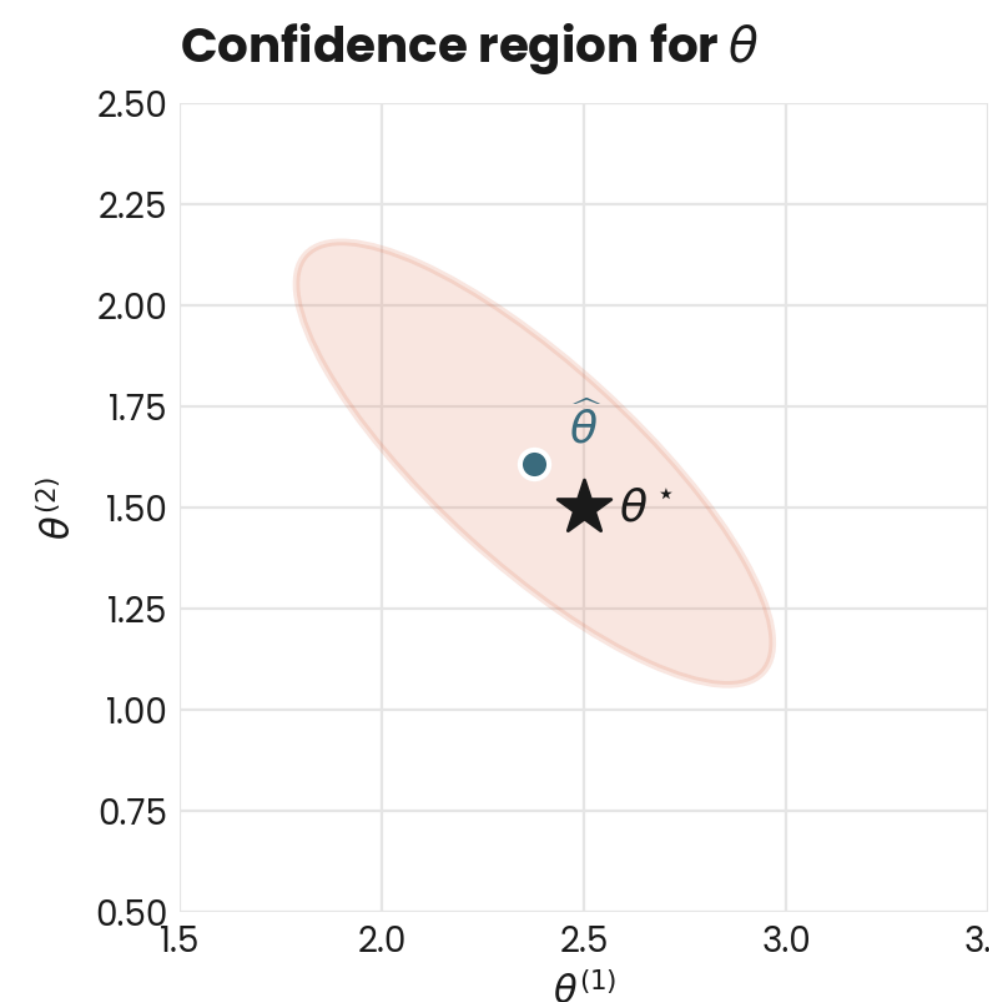
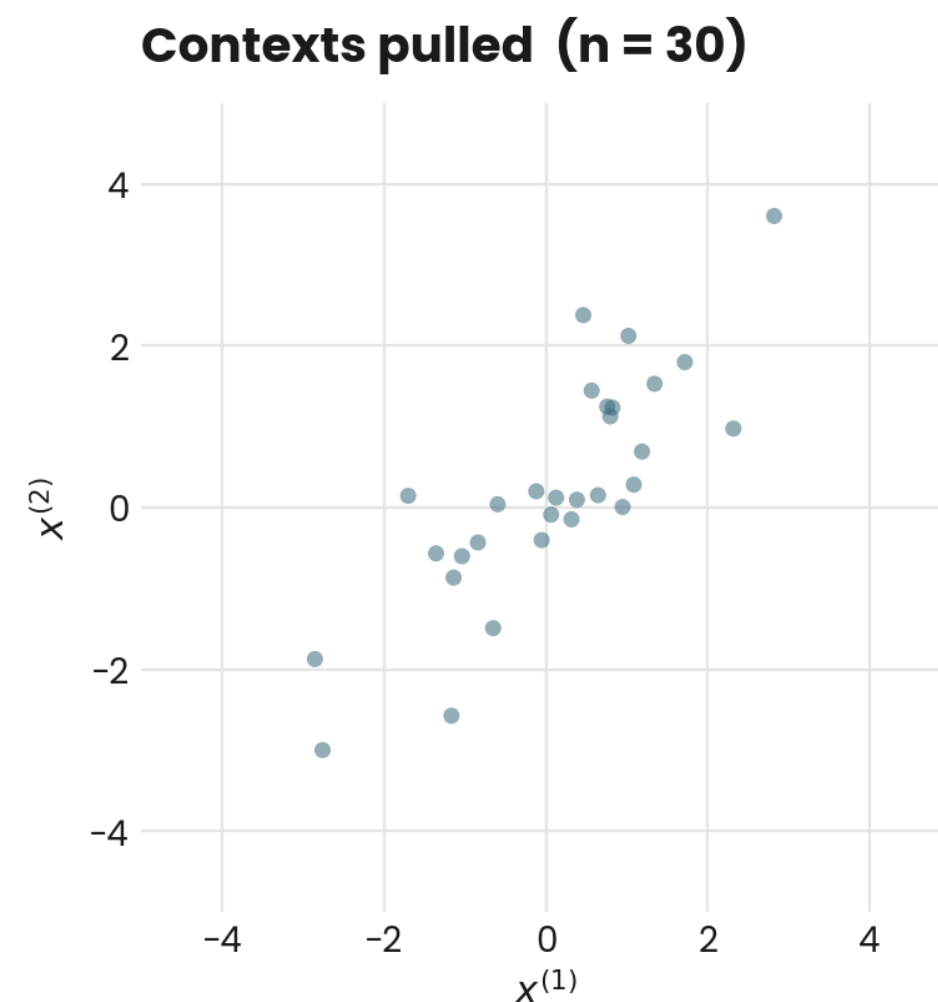
Isotropic exploration — few samples



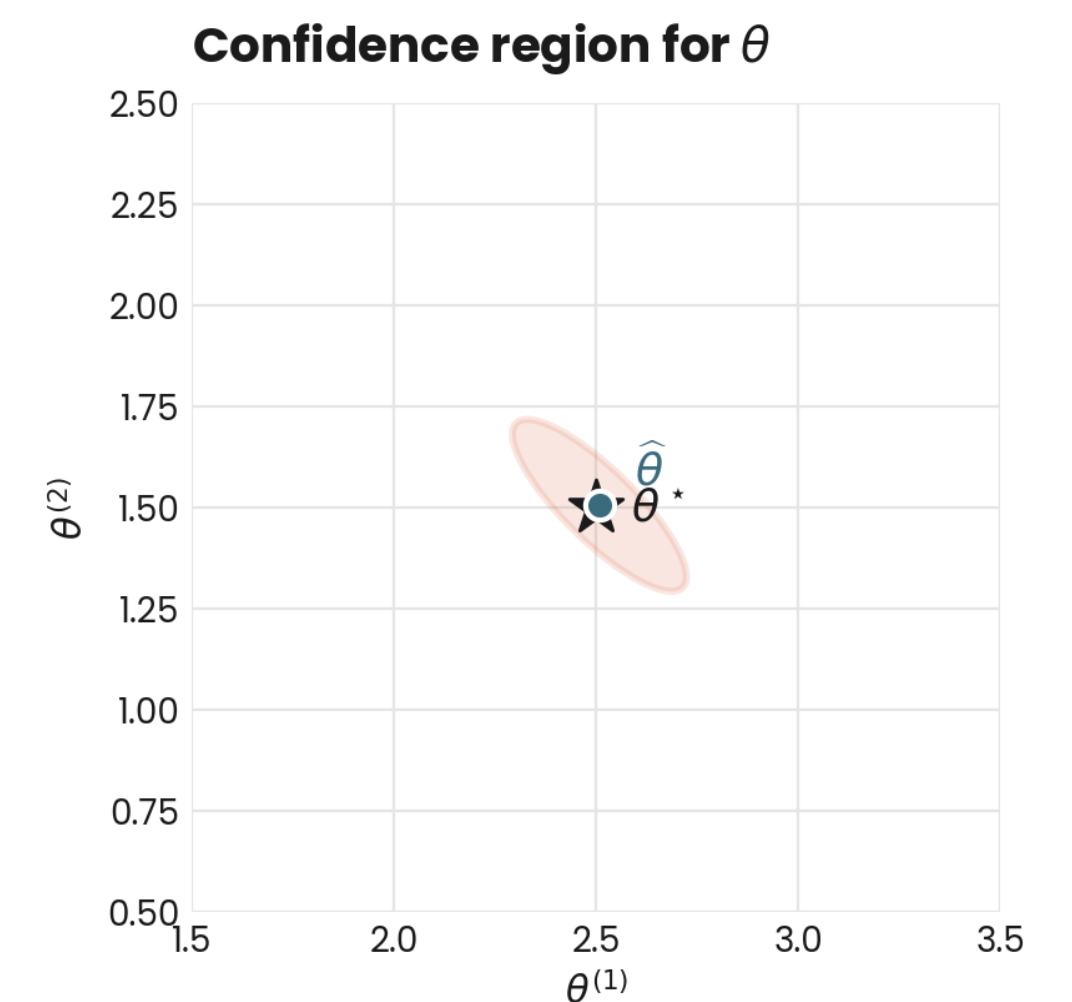
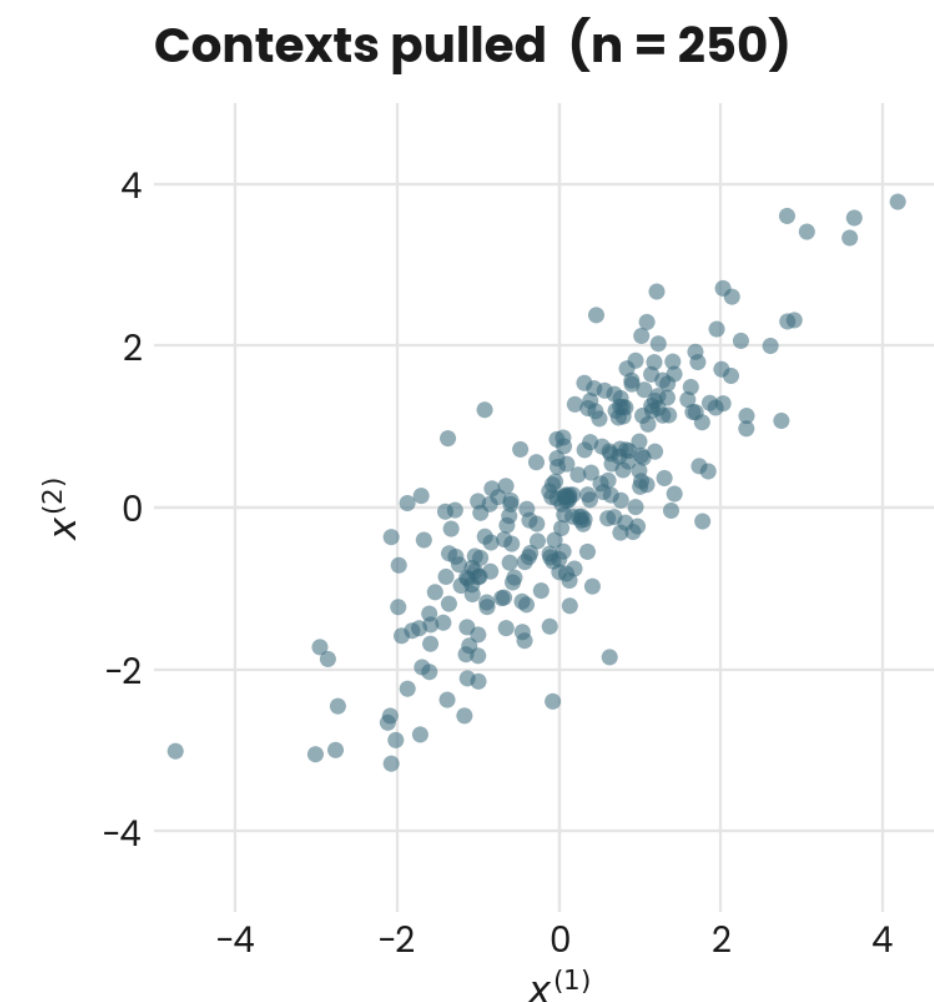
Isotropic exploration — many samples



Anisotropic exploration — few samples



Anisotropic exploration — many samples



# The LinUCB Algorithm

- $\mathcal{C}_t = \left\{ \theta \in \mathbb{R}^d : \|\theta - \hat{\theta}_{t-1}\|_{V_{t-1}}^2 \leq \beta \right\}$

- $V_t = \lambda I + \sum_{s=1}^t x(s)x(s)^\top$

- $\hat{\theta}_t = V_t^{-1} b_t$

- $b_t = \sum_{s=1}^t x(s)y(s)$

## LinUCB Algorithm

For  $t = 1, 2, \dots, T$ :

For all arms  $i \in [K]$ :

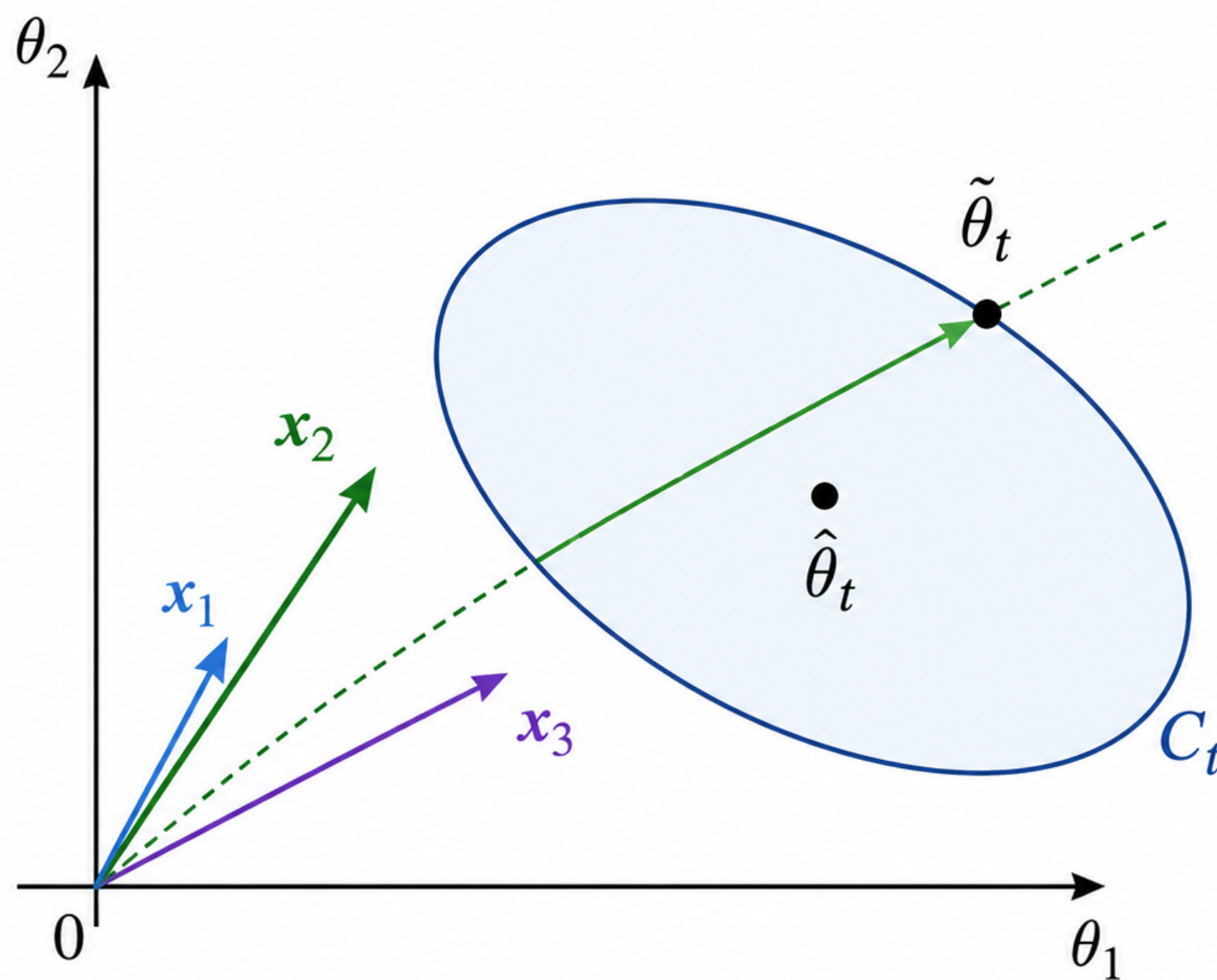
Compute  $UCB_t(i) = \max_{\theta \in \mathcal{C}_t} \langle \theta, x_i \rangle$

Play  $A_t = \arg \max_{i \in [K]} UCB_{t-1}(i)$

Observe  $y(t)$ , update  $\hat{\theta}_t, V_t, \mathcal{C}_t$



# The LinUCB Algorithm



$$UCB_t(\mathbf{x}_2) = \max_{\theta \in C_t} \langle \theta, \mathbf{x}_2 \rangle$$

Optimistic value of arm  $\mathbf{x}_2$ :  
best plausible reward over  $C_t$

$$A_t = \arg \max_a \max_{\theta \in C_t} \langle \theta, \mathbf{a} \rangle$$

## LinUCB Algorithm

For  $t = 1, 2, \dots, T$ :

For all arms  $i \in [K]$ :

Compute  $UCB_t(i) = \max_{\theta \in \mathcal{C}_t} \langle \theta, x_i \rangle$

Play  $A_t = \arg \max_{i \in [K]} UCB_{t-1}(i)$

Observe  $y(t)$ , update  $\hat{\theta}_t, V_t, \mathcal{C}_t$

**Coming Up Next:**

**How LinUCB works in practice**

